LiteTopK: Exploiting the Curse of Dimensionality for a Fused Indexer-TopK Kernel in Long-Context Sparse Attention

Ziqi Yin
Nanyang Technological University

Jianyang Gao
ETH Zurich

Peiqi Yin
The Chinese University of Hong Kong

Jiangneng Li
Nanyang Technological University

Gao Cong
Nanyang Technological University

Abstract

Indexer-TopK, the operation to compute the scores and select the top-k candidates, is widely used by sparse attention kernels in large language models and vector retrieval in recommendation systems and vector databases. However, existing GPU-based Indexer-TopK kernels like DeepSeek Sparse Attention (DSA) remain inefficient due to excessive global memory traffic, costly synchronization, and prohibitive memory overhead. In this work, we exploit the curse of dimensionality in high-dimensional spaces, where distances between high-dimensional vectors tend to concentrate within a narrow range, to design LiteTopK, a novel and efficient fused Indexer-TopK kernel. LiteTopK first samples a small subset of data to estimate query-data score ranges, then uses these estimates to partition candidate results into bins online. This organization allows the LiteTopK kernel to maintain a tight approximate threshold, write back only promising candidates, reduce unnecessary I/O, substantially lower memory overhead, and still preserve exact Top-k correctness. Experimental results show that LiteTopK accelerates the prefill stage of GLM 5.2 by $1.2\times$ in real-world deployment scenarios while incurring lower memory overhead.

1 Introduction

Indexer-TopK, which computes scores under a specific scoring function and selects the per-row top-k elements, has become a core primitive in modern machine learning systems, with applications in large language model (LLM) inference (Tang et al., 2024; Liu et al., 2025; Synk et al., 2025), recommendation systems (Khandagale et al., 2025; Gao et al., 2021), and information retrieval (Lee et al., 2023). Representative examples include:

1. In long-context LLM inference, attention computation becomes prohibitively expensive during the prefill stage as contexts grow to millions of tokens, motivating sparse-attention systems (Ribar et al., 2024; Chen et al., 2024; Liu et al., 2025; Synk et al., 2025; Tang et al., 2024) such as DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2026) to approximate full attention using only a small subset of tokens. Specifically, during the prefill stage, DSA treats the current text chunk as queries and the historical context as candidate keys and values. It computes and stores lightweight relevance scores between the preceding context and the chunk text, then applies top-k selection to identify the most relevant historical tokens for constructing sparse attention. While this design reduces the cost of full attention, its score-computation and selection pipeline itself becomes a major performance bottleneck. As shown in Figure 1, the DSA kernel, which includes both score computation and Top-k selection, dominates the prefill runtime of GLM 5.2 (GLM-5-Team et al., 2026) at a 1M context length, accounting for 83.7% of the total runtime.
2. Recommendation systems and information retrieval systems typically encode queries/users and candidate items into high-dimensional embeddings using deep learning models (Khandagale et al., 2025; Gao et al., 2021). During candidate retrieval, they use Indexer-TopK to identify the top-k

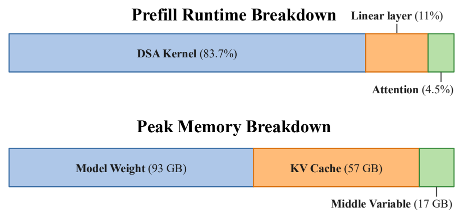


Figure 1: Breakdown of GLM-5.2 Prefill Runtime and Peak Memory Usage Across 8 B200 GPUs Using 8-Way Tensor Parallelism and Expert Parallelism.

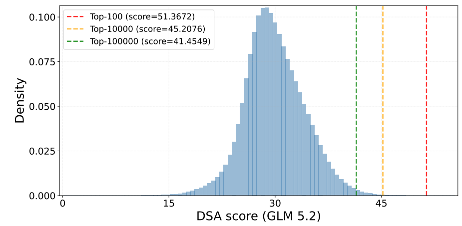


Figure 2: DSA Score distribution of GLM 5.2's first layer during prefill on the Wiki dataset.

candidates with the highest similarity scores, where k can be on the order of thousands. The retrieved candidates are then re-ranked by more sophisticated models.

Despite the broad applications, existing Indexer-TopK implementations remain inefficient on GPUs because they typically decouple score computation and selection into two stages: first, computing and materializing the full score matrix in high-bandwidth memory (HBM) and then performing a top-k selection, such as radix-select (Alabi et al., 2012). However, writing the entire score matrix back to HBM introduces two major overheads. First, it incurs substantial **memory overhead**. For example, the DSA kernel incurs a runtime memory overhead of 32 GB under a 1M-token prefill with a moderate chunk size (e.g., 8192 in vLLM Kwon et al. (2023)). This is because the kernel needs to keep the full score matrix before the top-k selection. Such runtime memory overhead reduces the portion of HBM for storing the KV cache, which is already tight in existing LLM serving systems. As shown in Figure 1, deploying GLM-5.2 on 8 B200 GPUs with 0.95 memory utilization limit provides 170 GB per GPU, while the model weights, KV cache, and intermediate activations together consume 167 GB, leaving less than 3 GB of headroom. In practice, this forces vLLM (Kwon et al., 2023) to further split the 8192-token chunk into smaller sub-chunks during prefill, such as 128, in order to reduce the HBM footprint and avoid out-of-memory errors, but at the cost of lower prefill throughput[1]. Second, it incurs significant **latency overhead**, since the score matrix must be written back to HBM and subsequently read multiple times during Top-k selection, leading to significant latency. The same issue is also noted in the vLLM blog[2], which observes that "a clear challenge is that at high batch size with long context, the logits tensor is materialized before running a row-wise top-k."

To address this issue, we study the distribution of the scores in sparse attention and observe that these scores consistently show a concentration phenomenon, which is brought by the curse of dimensionality in a high-dimensional space (Weber et al., 1998; Indyk & Motwani, 1998). In particular, in high-dimensional vector spaces, similarity scores between vectors, such as Euclidean distance and inner product, tend to concentrate within a narrow range while exhibiting an extremely long-tailed distribution, a phenomenon widely observed in real-world datasets (Yin et al., 2026; Bruch, 2024). Since sparse-attention scores are typically derived from the similarity scores between high-dimensional vectors, this concentration effect also appears in sparse attention. For example, DSA applies ReLU-based weighting to inner-product scores, resulting in a distribution that exhibits the same narrow-range concentration and long-tail behavior. As shown in Figure 2, for the first layer of GLM-5.2 under a 1M-token context, DSA scores are concentrated between 25 and 35, while scores greater than or equal to 45 are extremely rare, with only about 10K such scores.

Leveraging this pattern, we propose LiteTopK, a kernel that follows a sample-filter-select framework. It initializes a tight threshold and dynamically updates it to filter out unpromising candidates, effectively reducing memory overhead and I/O cost. Specfcially, in the sample phase, LiteTopk exploits the observation that, in sparse attention, adjacent tokens attend to the same prefill tokens and often carry related semantics, making their score distributions likely to be similar. Therefore, we uses the top-k' scores from the previous chunk ($k' = 3k$ for DSA), as a lightweight sample to estimate the score distribution of the current chunk. Then, we apply equal-width quantization to partition the estimated score range into non-overlapping sub-ranges, referred to as bins, initializes a histogram to

[1] https://github.com/vllm-project/vllm/pull/36178

[2] https://vllm.ai/blog/2025-09-29-deepseek-v3-2

count the number of candidates in each bin, and identify the bin containing the local Top-k element as the threshold bin.

In the filtering phase, we compute scores for all candidates and dynamically maintain a threshold bin. The distance upper bound of this bin is then used to filter out unpromising candidates, thereby substantially reducing write-back pressure. To prevent histogram updates from slowing down the overall GPU-parallel execution, we assign idle warps to update the histogram via overwrite writes instead of lock-based operations, since a stale and slightly loose threshold does not affect correctness or efficiency. To minimize the overhead of computing bin IDs, we replace the original DSA-score computation with bin-ID computation without introducing additional latency, directly store the intermediate bin IDs in floating-point format, and restore the corresponding scores only when returning the final results. To reduce write overhead, we maintain a list in warp-local shared memory and flush it in batches. In the selection phase, selection is required only within the threshold bin, whereas candidates from all preceding bins are directly written to the output, thereby substantially reducing I/O cost and selection overhead.

In summary, this work makes the following contributions:

- We are the first to observe and characterize the distance concentration phenomenon in sparse attention, trace its origin to the well-known curse of dimensionality in high-dimensional spaces, and further exploit this property to improve sparse attention kernel design.
- We propose LiteTopK, the first indexer-TopK fused kernel, which substantially reduces write-back memory pressure and achieves a $1.24\times$ speedup over the fastest existing DSA kernel. Notably, the design principle of our kernel can also be extended to other sparse attention kernels.
- In a real-world deployment on 8 B200 GPUs, LiteTopK achieves a $1.2\times$ end-to-end speedup on GLM-5.2 while using less memory. The source code of our implementation is available at `https://github.com/Heisenberg-Yin/LiteTopK`.

2 Related Work

2.1 Sparse Attention

As million-token context windows get widely deployed into real-world production environments, sparse attention (DeepSeek-AI, 2026; Tang et al., 2024; Ribar et al., 2024) has become critical to reduce the I/O and compute costs of full attention in both the prefill and decode stages of LLM inference. A representative example is DeepSeek Sparse Attention (DSA) (Liu et al., 2025), a natively trained sparse attention module, which has become the foundation for massively deployed models such as DeepSeek-V4 (DeepSeek-AI, 2026) and GLM-5.2 (GLM-5-Team et al., 2026). DSA follows a score-then-select paradigm. It develops a lightweight indexer, which stores an indexer key vector $k_s \in \mathbb{R}^d$ in FP8 for every preceding token s. For a query token t, it derives multiple indexer query vectors $q_{t,j} \in \mathbb{R}^d, j = 1, ..., H$ and compute index scores based on the following expression

$$I_{t,s} = \sum_{j}^{H} w_{t,j} \, \mathrm{ReLU}\big((q_{t,j})^\top k_s\big) \,. \tag{1}$$

where $w_{t,j}$ is a query-dependent weight used to aggregate the per-head scores. A per-query top-k selection with $k = 2,048$ is then applied to this aggregated score $I_{t,s}$, producing a subset of tokens for attention computation.

Another class of sparse-attention methods adopts an MHA/GQA-style design, performing score-then-select independently for each attention head. These methods are typically training-free and operate directly on off-the-shelf checkpoints, differing mainly in how per-head relevance scores are computed, such as using exact query–key inner products (Gupta et al., 2021; Yang et al., 2025) or approximate scores from reduced representations, such as salient query channels (Ribar et al., 2024) and product quantization (Zhang et al., 2025). However, training-free methods typically introduce non-negligible accuracy degradation. In this study, we select the representative sparse-attention method DSA for the prefill stage, while LiteTopK is broadly applicable to other score-then-select sparse-attention paradigms as well.

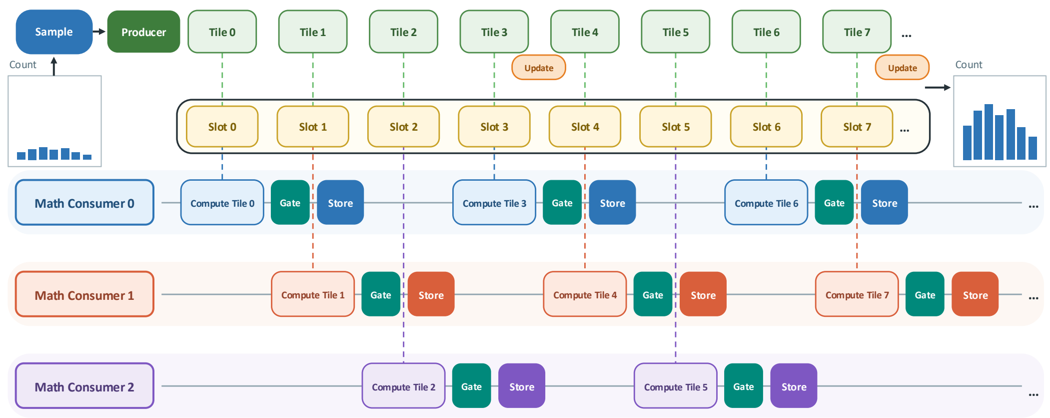


Figure 3: Illustration of the LiteTopK Kernel.

2.2 Top-k selection

According to (Zhang et al., 2023), existing parallel top-k selection algorithms on GPUs can be categorized into four groups: sorting-based methods (Huang et al., 2009), partial-sorting methods (Douze et al., 2025; Johnson et al., 2019; Shanbhag et al., 2018; Zhang et al., 2023), partition-based methods (Dashti et al., 2013; Komarov et al., 2014; Zhang et al., 2023), and hybrid methods (Gaihre et al., 2021; Zhang et al., 2023). Among these methods, radix select is widely regarded as one of the most efficient approaches and has been broadly adopted in industrial systems, including PyTorch and NVIDIA's RAFT library. However, these methods typically assume that all scores have already been explicitly materialized in memory before selection is invoked.

More recently, (Yang et al., 2026) explored a more I/O-efficient direction, which attempts to fuse score computation and top-k selection into a single kernel to mitigate the limitations discussed in the Introduction. It maintains a local top-k set for each block on-chip and reduces these partial results to the final top-k, thus avoiding full score-matrix writes to HBM. However, this approach is constrained by limited per-block register capacity: high efficiency is achieved only for very small k values, e.g., k = 10. As k increases, efficiency drops sharply due to register pressure and costly synchronization, and larger values of k, such as $k \geq 1024$, are not supported. This is consistent with our experimental findings: its performance degrades sharply as k increases from 128 to 1,024 (see Section 4.2). This substantially limits its application scenarios: DSA often requires retrieving the top-2,048 entries.

3 LiteTopK

3.1 Problem Definition, Backgroud, and Overview

Problem Definition. Given a query set Q and a candidate set X, our goal is to exactly identify, for each query, the k candidates with the highest scores under a scoring function f:

$$\mathrm{TopK}(q) = \underset{x \in X}{\arg\operatorname{topk}}\, f(q, x), \qquad q \in Q. \tag{2}$$

Sparse attention, where f is an indexer score such as the DSA score in Equation 1, is the focus of this paper. We also consider other scenarios such as k-nearest-neighbor (k-NN) search.

Background. Modern GPUs provide two kinds of arithmetic units. Tensor Cores are dedicated matrix-multiplication units that deliver the vast majority of the peak FLOPs and dominate the throughput of GEMM-style kernels. CUDA cores are general-purpose units that execute ordinary scalar instructions and handle element-wise post-processing such as activation, scaling, and aggregation. Their basic floating-point instruction is the fused multiply-add (FFMA), which computes $a \cdot b + c$ in a single instruction, where the addend c also serves as the accumulator across a chain of FFMAs. Crucially, the two kinds of units have separate pipelines and execute concurrently: while Tensor Cores process matrix-multiplication instructions, the warp scheduler can issue independent CUDA-core instructions in parallel. In kernels whose scores are produced by Tensor Cores, the CUDA cores are therefore largely idle, and a small amount of per-score scalar post-processing can be overlapped with the ongoing matrix computation, so it stays off the critical path as long as its cost remains well below the Tensor-Core work per tile. In contrast, when the score itself is finalized on CUDA cores, as in DSA, whose ReLU-weighted aggregation (Equation 1) runs on CUDA cores, the CUDA-core path is itself the bottleneck, and every additional instruction on it adds latency.

On top of these units, GPU matrix-multiplication kernels, and the sparse-attention indexers built on top of them, are organized at the level of Cooperative Thread Arrays (CTAs). Each CTA consists of multiple warps of 32 threads, specialized into two roles: a producer warp dedicated to data movement, which continuously loads data tiles from global memory into shared memory, and consumer (math) warps, which fetch tiles from shared memory and perform the matrix multiplication. The two roles are pipelined: while the producer moves the next tile into shared memory, one consumer loads the current tile into registers and another computes on the previous tile, thereby reducing idle time and improving hardware utilization. Two properties of this pipeline are central to our design: (i) when scores are produced by Tensor Cores, the consumers' CUDA cores have spare cycles, and (ii) the producer warp becomes idle once it has issued all fetch commands for the current stage. LiteTopK places all of its extra operations in exactly these gaps.

Overview. LiteTopK fuses top-k selection into the scoring kernel through a sample-filter-select workflow, illustrated at the CTA level in Figure 3. (1) Sample: before the main loop starts, we score a small sample of X to estimate the score range, partition this range into m equal-width bins, initialize a histogram of bin counts, and set an initial threshold bin, the bin contain the current k-th largest score. (2) Filter: in the main loop, as soon as a batch of scores is produced, each score is mapped to its bin ID and compare against the threshold bin ID. Only passing candidates are kept in a per-warp list and flushed in batches to a shared-memory candidate buffer later, while the histogram counts are updated along the way. (3) Refresh: we periodically use an idle warp to recompute the threshold bin from the histogram, so the gate tightens as the scan proceeds. (4) Select: after the scan, candidates in bins above the final threshold bin are written directly to the output, and a tail selection is performed only within the threshold bin.

3.2 METHOD

We now detail the four stages in turn.

Sampling. The sample serves two purposes: it must span the score range so that the bins are well scaled, and it should contain scores close to the true top-k threshold so that the initial gate is tight. An overly coarse sample degrades the effectiveness of filtering. For sparse attention, we exploit the observation that prefix keys and values are shared across chunks, and that adjacent chunks naturally exhibit similar semantics and therefore tend to attend to the same prefix tokens. We therefore reuse the k' tokens (e.g., $k' = 3k$ for DSA) that occur most frequently in the top-k results of the previous chunk as the sample for the current chunk. Such tokens are likely to remain high-scoring, yielding a tight initial threshold. Since a prefill chunk contains thousands of queries (e.g., 8,192 tokens), the cost of selecting the top-k' tokens is amortized over thousands of scoring computations, and the selection is executed asynchronously, making it negligible in practice. For k-NN search, whose score distribution is also concentrated but exhibits no such temporal structure, random sampling suffices.

We then score the k' sampled candidates to obtain the minimum and maximum sample scores $s_{\min}$ and $s_{\max}$, and apply equal-width quantization to partition $[s_{\min}, s_{\max}]$ into m bins of width $\Delta = (s_{\max} - s_{\min})/m$: a score s falls into bin $\lfloor (s - s_{\min}) \cdot \delta \rfloor$. Here, we store the reciprocal $\delta = 1/\Delta$ so that bin IDs are computed with a multiplication rather than a division, which is more efficient on GPU. The histogram is initialized with the bin counts of the sample, and the threshold bin is initialized as the bin containing the k-th largest sampled score. During filtering, a candidate passes the gate if and only if its bin ID is no smaller than the threshold bin ID, i.e., its score is at least the lower edge of the threshold bin. This initialization is conservative: since the sample is a subset of X, its k-th largest score cannot exceed the k-th largest score over all of X, so the true top-k threshold lies in or above the threshold bin and every true top-k candidate passes the gate.

Filtering. Filtering is fused into the existing scoring pipeline without changing its scoring semantics, and involves two concerns: computing bin IDs at negligible cost, and writing back the passing candidates efficiently. For the first concern, consider how scores are produced. When scores are produced directly by Tensor Cores, as in inner-product-based sparse-attention kernels Yang et al. (2025), the bin ID is computed on CUDA cores immediately after each score is produced, adding only a few scalar instructions per score (an FFMA for the bin ID and a comparison for the gate). Since such kernels are Tensor-Core-bound and their CUDA cores are largely idle (Section 3.1), this post-processing overlaps with the matrix multiplication of subsequent tiles and introduces negligible overhead. The kernel simply outputs each score together with its bin ID. DSA is more challenging: its score is finalized on CUDA cores, so any additional arithmetic there directly adds latency. We therefore fold the bin mapping into the score computation itself. The key observation is that the bin-space score $B_{t,s} = (I_{t,s} - s_{\min}) \cdot \delta$ is an affine transform of the DSA score, and can be produced by the existing computation:

$$B_{t,s} = (I_{t,s} - s_{\min}) \cdot \delta = \sum_j w'_{t,j} \, \mathrm{ReLU}(q_{t,j})^\top \, k_s - s_{\min} \cdot \delta, \qquad w'_{t,j} = w_{t,j} \cdot \delta. \tag{3}$$

Since δ remains fixed throughout the scan, the weights are pre-scaled ($w'_{t,j} = w_{t,j}\delta$) once, before being loaded into registers. Moreover, the weighted sum is already computed with a chain of FFMA instructions whose accumulator is normally initialized to zero. We simply initialize it to the affine offset $-s_{\min} \cdot \delta$ instead, which is effectively free. The FFMA chain then directly outputs $B_{t,s}$, from which the bin ID follows, adding no instructions to the critical path. For each surviving candidate we store the floating-point bin-space score $B_{t,s}$ and discard the raw score. Because the transform is a fixed invertible affine map, the raw score can be recovered from $s_{\min}$ and δ at the final output stage, making the transformation lossless.

The second concern is write-back. Candidates that pass the gate are not written to the shared-memory candidate buffer immediately. Instead, each warp first collects them in a small unordered staging list and flushes the list only when it reaches a predefined length, such as 32 or 64 entries, using all 32 threads of the warp concurrently. This batching has two benefits. First, it reduces atomic contention on the buffer tail: the tail pointer is updated once per flush rather than once per candidate, cutting the contended updates to $1/32$ or $1/64$ of the original frequency. Second, it better utilizes warp-level parallelism: all 32 lanes participate in a batched flush, whereas immediate per-candidate writes activate a single lane and leave the rest idle. In addition, each passing candidate issues one atomic increment to its histogram bin. These increments are fire-and-forget: their results are consumed only by the periodic threshold refresh described next, never on the critical path, so they do not block the main execution flow.

Threshold Refresh. As the histogram grows, the gate can tighten beyond its initialization. Periodically, an idle warp, typically the producer after it has issued all fetch commands, or any other naturally idle warp, recomputes the threshold bin by accumulating histogram counts from the highest bin downward until the cumulative count reaches k, and publishes it with a plain overwrite write instead of an atomic update. This requires no synchronization and therefore preserves the existing parallel execution order on the GPU. A stale threshold merely loosens the gate, admitting a few extra candidates, but are rare under score concentration. Threshold maintenance thus stays entirely off the critical path, hidden in idle cycles.

Top-k Selection. When the scoring process completes, all surviving candidates reside in the candidate buffer together with their bin-space scores, and the final threshold bin is known. By the refresh rule, the bins above the threshold bin jointly contain fewer than k candidates. These candidates are written directly to the output. The remaining slots are filled by a tail selection restricted to the candidates inside the threshold bin, whose number is small under score concentration. Raw scores are recovered from the stored bin-space values via the inverse affine map. Compared with decoupled designs that run a full top-k selection, such as radix select, over all $|X|$ materialized scores, LiteTopK selects among only the threshold-bin candidates, which substantially reduces the selection cost.

4 Experiments

4.1 Experimental Setup

Evaluation Overview. We organize our evaluation around two application domains. For long-context Large language model serving, we first evaluate LiteTopK in an end-to-end deployment of GLM-5.2's prefill stage, and then isolate the sparse-attention kernel to characterize its latency and memory consumption under controlled configurations. For large-scale vector retrieval, we evaluate whether the benefits of LiteTopK generalize beyond sparse attention workloads. Together, these experiments

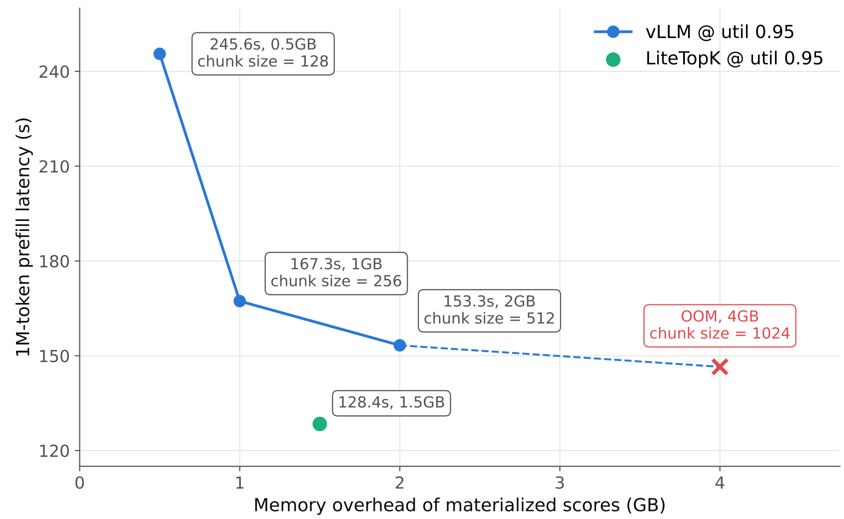


Figure 4: Memory-overhead vs End2End latency - 1M prefill, GLM-5.2-FP8, 8xB200, TP8, MTP on.

assess the application-level effectiveness, kernel-level efficiency, and cross-domain generality of LiteTopK.

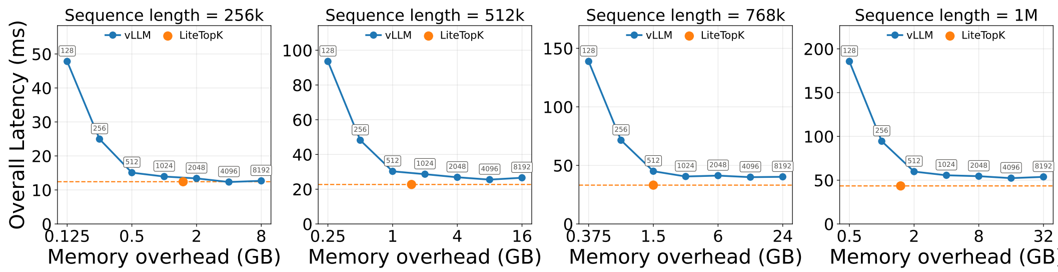
End-to-End Long-Context Prefilling. We first evaluate LiteTopK in an end-to-end deployment of GLM-5.2 (GLM-5-Team et al., 2026), with prefill performed at a 1M-token context length, a model with a native context length of one million tokens. We serve the FP8 model using vLLM 0.23 (Kwon et al., 2023) on eight NVIDIA B200 GPUs and replace the original DSA kernel with LiteTopK only for context lengths starting from 256K, while leaving the rest of the serving stack unchanged to isolate the impact of our method. Following the official vLLM configuration, we enable eight-way tensor parallelism and expert parallelism, compile the model with `torch.compile` and CUDA graphs, enable asynchronous scheduling to emulate realistic serving workloads, and set the prefill chunk size to 8,192 tokens.

The default configuration with `gpu_memory_utilization` set to 0.90 results in an out-of-memory error. We therefore follow the recommended configuration and increase it to 0.95. For the baseline, which materializes intermediate score logits, we control memory consumption through additional prefill sub-chunking. Specifically, we vary the maximum score-logit footprint from 512 MB to 1 GB, 2 GB, and 4 GB. A larger budget reduces the amount of sub-chunking and can reduce latency, but also increases peak memory consumption and eventually causes the baseline to run out of memory. LiteTopK, in contrast, avoids materializing these large intermediate score logits and can therefore process the full 8,192-token prefill chunk without additional sub-chunking. We report end-to-end prefill latency and peak auxiliary memory consumption to characterize this latency-memory trade-off.

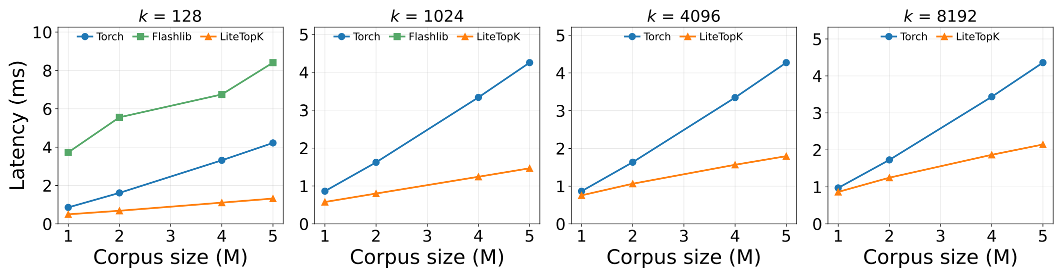
Sparse-Attention Kernel Benchmark. We next isolate the DSA workload to examine the source of the end-to-end improvements. We compare LiteTopK against two implementations. The first is the official DSA implementation[3], which uses a custom CUDA kernel for score computation and invokes PyTorch's `topk` operator for Top-k selection. The second, denoted as vLLM, is an optimized implementation for the NVIDIA Blackwell architecture that replaces PyTorch's `topk` operator to reduce unnecessary I/O traffic.

We construct the benchmark inputs from the first-layer attention activations of GLM-5.2, using text sampled from Wikipedia (Merity et al., 2016). Following the DSA evaluation setup, we consider context lengths of 256K, 512K, 768K, and 1M tokens, vary the prefill chunk size from 128 to 8,192 tokens, and fix k to 2,048. For each configuration, we report the aggregate kernel latency for processing 8,192 query tokens. When the queries are divided into multiple chunks, the reported latency is the sum of the latency across all chunks. We additionally measure peak auxiliary memory consumption. For chunk sizes of 2,048, 4,096, and 8,192, both the official DSA and vLLM baselines require up to 8 GB, 16 GB, and 32 GB of additional memory, respectively. Although such memory

[3] `https://github.com/deepseek-ai/DeepGEMM`



(a) DSA



(b) MSMARCO

Figure 5: Runtime comparison between LiteTopK and existing methods on sparse-attention and retrieval workloads.

overheads are impractical for production deployment, we retain these configurations to characterize the best latency achievable by the baselines when sufficient memory is available.

Large-Scale Retrieval Benchmark. Finally, we evaluate LiteTopK on large-scale retrieval to determine whether its efficiency gains generalize beyond sparse attention. We construct a corpus of five million passages sampled from MSMARCO-V2.1[4], a widely used text-retrieval dataset. Each passage is encoded as a 768-dimensional vector using the Snowflake Arctic-embed-m-v1.5 model (Merrick, 2024). We construct the query set Q by randomly sampling 1,000 passages and evaluate corpus sizes of 1M, 2M, 4M, and 5M vectors. We vary k over 128, 1,024, 4,096, and 8,192 and use a batch size of 64 to emulate concurrent retrieval requests. Because the DSA and vLLM baselines above are specialized for attention workloads, we instead compare LiteTopK with Flashlib on this task and report Flashlib results for the configurations it supports.

Evaluation Metrics. LiteTopK preserves the input–output semantics of the original Top-k implementations; therefore, our evaluation focuses on systems efficiency. For the end-to-end serving experiment, we report prefill latency and peak auxiliary memory consumption. For the isolated sparse-attention and retrieval benchmarks, we report aggregate kernel latency and peak auxiliary memory consumption under each workload configuration.

Implementation. All our implementations are written in CUDA C++ using CuTe/CUTLASS, while the baselines are evaluated base on their official implementations. The experiments are conducted on an NVIDIA B200 GPU with 180 GB of memory using CUDA 12.8, and we additionally conduct experiments on H100 GPUs. All reported running times are averaged over 20 runs.

4.2 Experimental Results

Results Overview. We first evaluate the end-to-end benefits of LiteTopK in long-context model serving, and then analyze the underlying kernel-level improvements and their portability across GPU architectures. Finally, we evaluate LiteTopK on large-scale retrieval to examine whether its advantages generalize beyond sparse-attention workloads.

[4]`https://huggingface.co/datasets/Snowflake/msmarco-v2.1-snowflake-arctic-embed-m-v1.5`

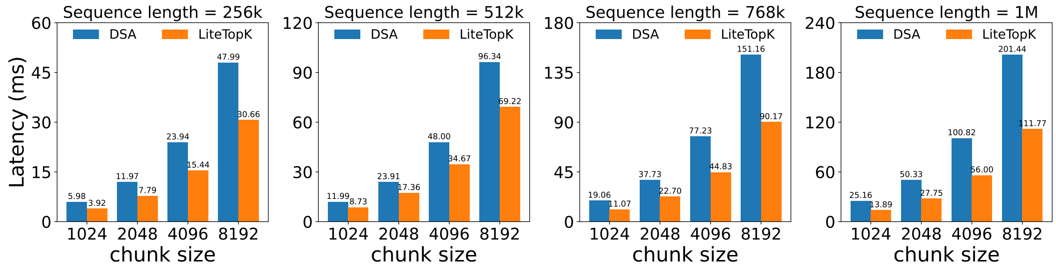


Figure 6: Runtime comparison between LiteTopK and the official DSA implementation on NVIDIA H100 GPUs. The prefill chunk size is fixed to 8,192 tokens.

End-to-End Long-Context Prefilling. Figure 4 reports the end-to-end prefill latency and peak auxiliary memory consumption of GLM-5.2 in a realistic serving configuration. The vLLM baseline reduces latency by increasing its internal prefill sub-chunk size, but this improvement comes at the cost of substantially higher memory consumption and eventually results in an out-of-memory error. Its fastest feasible configuration uses a sub-chunk size of 512 and requires 153.3 s to process the 1M-token input, with 2.0 GB of auxiliary memory. LiteTopK completes the same workload in 128.4 s while consuming only 1.5 GB, corresponding to a $1.20\times$ speedup with 25% lower auxiliary memory consumption.

The reported memory consumption of LiteTopK includes a deliberately conservative candidate buffer with a capacity of $12k$, corresponding to 24,576 candidates for $k = 2,048$. In practice, the number of candidates retained after filtering is typically substantially smaller than this capacity. The reported 1.5 GB footprint therefore represents a conservative implementation point and can be further reduced through tighter candidate-buffer allocation. These results demonstrate that avoiding score-logit materialization allows LiteTopK to translate its kernel-level efficiency into measurable end-to-end latency and memory improvements.

Sparse-Attention Kernel Performance. Figure 5a compares LiteTopK with the official DSA implementation and the optimized vLLM kernel on NVIDIA B200 GPUs. Under the same chunk size of 8,192 and a context length of 1M, LiteTopK reduces the aggregate kernel latency from 146.6 ms to 43.4 ms relative to DSA, yielding a $3.38\times$ speedup. It also outperforms the Blackwell-optimized vLLM implementation by $1.24\times$, reducing latency from 53.8 ms to 43.4 ms.

This equal-chunk-size comparison isolates the computational efficiency of the kernels, but the baseline configurations require 32 GB of auxiliary memory at a chunk size of 8,192, making them difficult to use in memory-constrained deployments. When vLLM instead uses more practical chunk sizes of 512 and 1,024, its latency increases to 59.8 ms and 55.6 ms, respectively. Because LiteTopK has substantially lower memory overhead, it can retain the 8,192-token chunk size and achieve deployment-oriented speedups of $1.38\times$ and $1.28\times$ over these two vLLM configurations.

The advantage of LiteTopK also increases with context length. At a chunk size of 8,192, LiteTopK reduces the DSA latency from 36.6 ms to 12.6 ms at a context length of 256K, corresponding to a $2.90\times$ speedup. At a context length of 1M, the speedup increases to $3.38\times$. This widening performance gap indicates that LiteTopK scales more favorably as the number of candidate tokens grows.

Portability to NVIDIA H100. We further evaluate LiteTopK on NVIDIA H100 GPUs, as shown in Figure 6, while fixing the prefill chunk size to 8,192 tokens. The vLLM baseline evaluated above relies on optimizations specific to the Blackwell architecture and is therefore unavailable on H100; accordingly, we compare LiteTopK with the official DSA implementation. LiteTopK retains a similar acceleration trend across the evaluated context lengths, indicating that its benefits are not limited to Blackwell GPUs.

Large-Scale Retrieval Performance. Figure 5b reports the results on the MSMARCO retrieval workload. LiteTopK consistently outperforms FlashLib across the evaluated values of k. Even at $k = 128$, FlashLib is substantially slower than LiteTopK, and the performance gap widens further as k increases. This degradation is consistent with the increasing cost of maintaining FlashLib's ordered on-chip data structure: as k grows, both the size of the maintained state and the cost of updating it increase. Consequently, FlashLib is better suited to very small-k workloads, whereas LiteTopK remains efficient for the large-k configurations required by our sparse-attention and retrieval workloads.

5 Conclusion and Discussion

This paper proposes an Indexer–TopK fused operator to accelerate the prefill stage of sparse attention in long-context inference. A promising direction for future work is to extend this approach to the decode stage, particularly by integrating it with speculative decoding.

References

Tolu Alabi, Jeffrey D Blanchard, Bradley Gordon, and Russel Steinbach. Fast k-selection algorithms for graphics processing units. *Journal of Experimental Algorithmics (JEA)*, 17:4–1, 2012.

Sebastian Bruch. *Foundations of vector retrieval*, volume 1. Springer, 2024.

Renze Chen, Zhuofeng Wang, Beiquan Cao, Tong Wu, Size Zheng, Xiuhong Li, Xuechao Wei, Shengen Yan, Meng Li, and Yun Liang. Arkvale: Efficient generative llm inference with recallable key-value eviction. *Advances in Neural Information Processing Systems*, 37:113134–113155, 2024.

Ali Dashti, Ivan Komarov, and Roshan M D'Souza. Efficient computation of k-nearest neighbour graphs for large high-dimensional data sets on gpu clusters. *Plos one*, 8(9):e74113, 2013.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.

Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, 2025.

Anil Gaihre, Da Zheng, Scott Weitze, Lingda Li, Shuaiwen Leon Song, Caiwen Ding, Xiaoye S Li, and Hang Liu. Dr. top-k: delegate-centric top-k on gpus. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–14, 2021.

Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. Learning an end-to-end structure for retrieval in large-scale recommendations. In *Proceedings of the 30th ACM international conference on information & knowledge management*, pp. 524–533, 2021.

GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye

Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. Glm-5: from vibe coding to agentic engineering, 2026. URL `https://arxiv.org/abs/2602.15763`.

Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pp. 39–52, 2021.

Bonan Huang, Jinlan Gao, and Xiaoming Li. An empirically optimized radix sort for gpu. In *2009 IEEE international symposium on parallel and distributed processing with applications*, pp. 234–241, 2009.

Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pp. 604–613, 1998.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE transactions on big data*, 7(3):535–547, 2019.

Sujay Khandagale, Bhawna Juneja, Prabhat Agarwal, Aditya Subramanian, Jaewon Yang, and Yuting Wang. Interactrank: Personalized web-scale search pre-ranking with cross interaction features. In *Companion Proceedings of the ACM on Web Conference 2025, WWW 2025, Sydney, NSW, Australia, 28 April 2025 - 2 May 2025*, pp. 287–295, 2025.

Ivan Komarov, Ali Dashti, and Roshan M D'Souza. Fast k-nng construction with gpu-based quick multi-select. *PloS one*, 9(5):e92409, 2014.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Zhao. Rethinking the role of token retrieval in multi-vector retrieval. *Advances in Neural Information Processing Systems*, 36:15384–15405, 2023.

Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

Luke Merrick. Embedding and clustering your data can improve contrastive pretraining. *arXiv preprint arXiv:2407.18887*, 2024.

Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: Bandwidth-efficient LLM inference. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, volume 235 of *Proceedings of Machine Learning Research*, pp. 42558–42583, 2024.

Anil Shanbhag, Holger Pirk, and Samuel Madden. Efficient top-k query processing on massively parallel hardware. In *Proceedings of the 2018 International Conference on Management of Data*, pp. 1557–1570, 2018.

Ryan Synk, Monte Hoover, John Kirchenbauer, Neel Jain, Alex Stein, Manli Shu, Josue Melendez Sanchez, Ramani Duraiswami, and Tom Goldstein. Exploiting sparsity for long context inference: Million token contexts on commodity gpus. *arXiv preprint arXiv:2502.06766*, 2025.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.

Roger Weber, Hans-Jörg Schek, and Stephen Blott. A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. In *VLDB'98, Proceedings of 24rd International Conference on Very Large Data Bases, August 24-27, 1998, New York City, New York, USA*, pp. 194–205, 1998.

Lijie Yang, Zhihao Zhang, Zhuofu Chen, Zikun Li, and Zhihao Jia. Tidaldecode: Fast and accurate LLM decoding with position persistent sparse attention. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*, 2025.

Shuo Yang, Haocheng Xi, Yilong Zhao, Qiuyang Mang, Zhe Wang, Shanlin Sun, Kurt Keutzer, Joseph E. Gonzalez, Song Han, Chenfeng Xu, and Ion Stoica. Flashlib: Bringing flash magic to classical machine learning operators, 2026. URL `https://flashml-org.github.io/`.

Ziqi Yin, Gao Cong, Kai Zeng, Jinwei Zhu, and Bin Cui. Bbc: Improving large-k approximate nearest neighbor search with a bucket-based result collector. *arXiv preprint arXiv:2604.01960*, 2026.

Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kvcache for long context llm inference. *Proceedings of the ACM on Management of Data*, 3(3):1–30, 2025.

Jingrong Zhang, Akira Naruse, Xipeng Li, and Yong Wang. Parallel top-k algorithms on gpu: A comprehensive study and new methods. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–13, 2023.